**BỘ GIÁO DỤC VÀ ĐÀO TẠO**

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM**

**KHOA ĐIỆN – ĐIỆN TỬ**

# BÁO CÁO CUỐI KỲ MÔN HỌC

# HỆ THỐNG NHÚNG

# SLEEP MODE - LOW POWER

**MÃ MÔN HỌC: EMSY435664**

**SÁNG THỨ 6, ĐỢT 2, HỌC KÌ 1, NĂM HỌC 2025-2026**

**Thực hiện:**

| | |
|---|---|
| **Nguyễn Ngọc Trường Phi** | **23119187** |
| **Hà Quang Huy** | **23119147** |
| **Võ Trung Nguyên** | **23119180** |
| **Vũ Minh Quang** | **23119198** |

**Giảng viên hướng dẫn: ThS. Huỳnh Hoàng Hà**

Thành phố Hồ Chí Minh, Tháng 1 năm 2026

# MỤC LỤC

**MỤC LỤC** ..... **i**
**DANH MỤC HÌNH ẢNH** ..... **ii**
**CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG CUNG CẤP NĂNG LƯỢNG** ..... **1**
1.1. Các miền nguồn điện áp ..... 1
1.2. Bộ điều áp nguồn (Voltage Regulator) và các trạng thái hoạt động ..... 2
**CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VỀ SLEEP MODE** ..... **3**
2.1. Tổng quan về chế độ Sleep Mode ..... 3
2.2. Kiến trúc điều khiển năng lượng tại lõi Cortex-M3 ..... 4
*2.2.1. Thanh ghi điều khiển hệ thống (System Control Register - SCR)* ..... *4*
*2.2.2. Vai trò của bit SLEEPONEXIT trong việc quản lý chu trình ngủ* ..... *4*
2.3. Các cơ chế kích hoạt Sleep Mode ..... 5
*2.3.1. Cơ chế Sleep Now:* ..... *6*
*2.3.2. Cơ chế Sleep on Exit:* ..... *6*
2.4. Lệnh thực thi và phương thức đánh thức ..... 7
*2.4.1. Lệnh WFI (Wait For Interrupt)* ..... *7*
*2.4.2. Lệnh WFE (Wait For Event)* ..... *7*
**CHƯƠNG 3: THIẾT KẾ CHƯƠNG TRÌNH THỰC NGHIỆM SLEEP MODE** ..... **8**
3.1. Sơ đồ kết nối phần cứng ..... 8
3.2. Lưu đồ thuật toán ..... 9
3.3. Cấu hình CubeMX ..... 14
3.4. Phân tích chương trình ..... 15
**CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH** ..... **35**
4.1. Kết quả đo lường tiêu thụ dòng điện. ..... 35
4.2. Phân tích kết quả đo lường do điện tiêu thụ ..... 37
**CHƯƠNG 5: KẾT LUẬN VÀ ĐÁNH GIÁ** ..... **38**
5.1. Ưu và nhược điểm của Sleep Mode ..... 38
5.3. Các ứng dụng thực tế ..... 39

# DANH MỤC HÌNH ẢNH

Hình 1: Tổng quan về nguồn điện.....1

Hình 2: Các chế độ Sleep Mode .....5

Hình 3: Sơ đồ nối chân bằng Proteues.....8

Hình 4: Lưu đồ thuật toán chương trình Sleep Mode .....9

Hình 5: Lưu đồ thuật toán của chương trình Blink Led.....11

Hình 6:  Lưu đồ thuật toán của chương trình ngắt.....12

Hình 7: Cấu hình chân STM32F103C8T6.....14

Hình 8: Cấu hình xung CLK STM32F103C8T6 .....15

Hình 9: Đo dòng điện ở chế độ Run .....35

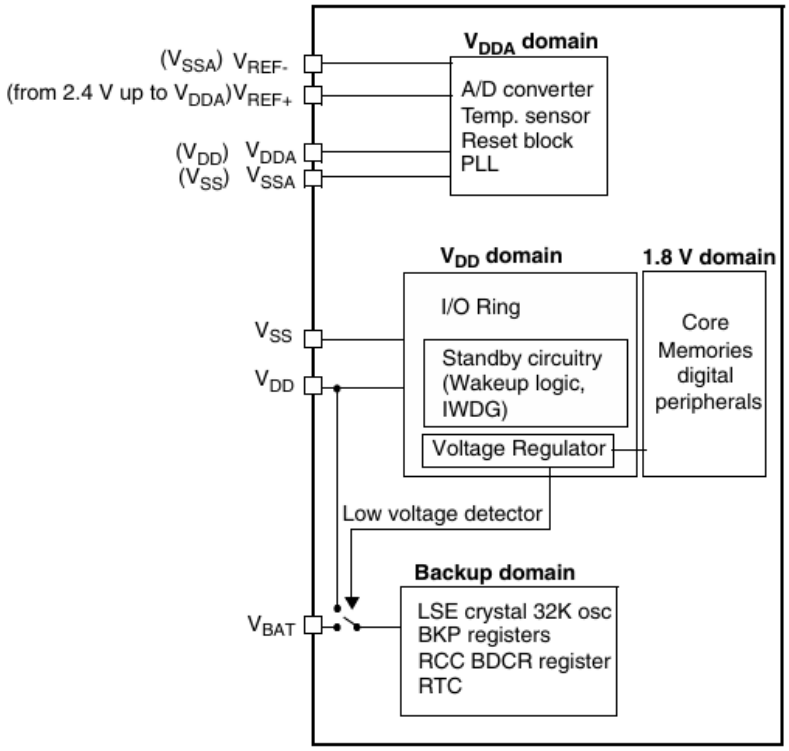
Hình 10: Đo dòng điện ở chế độ Sleep .....36

Hình 11: Đo dòng điện ở chế độ Sleep có GPIO ở mức High.....36

# CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG CUNG CẤP NĂNG LƯỢNG

## 1.1. Các miền nguồn điện áp

Hệ thống nguồn của STM32F103C8T6 được thiết kế chia tầng để tối ưu hóa công suất tiêu thụ:




Hình 1: Tổng quan về nguồn điện

**Miền nguồn chính ($V_{DD}$):** Có dải điện áp từ 2.0V đến 3.6V, cấp nguồn trực tiếp cho các chân I/O và bộ điều áp nguồn nội bộ.

**Miền nguồn Analog** ($V_{DDA}$): Có giá trị từ 2.4V đến 3.6V. Để cấp cho bộ ADC (tối thiểu 2.4V), các bộ dao động RC và PLL.

**Miền 1.8V:** Được cấp bởi bộ điều áp, cung cấp năng lượng cho lõi Cortex-M3, bộ nhớ và các ngoại vi kỹ thuật số.

**Miền dự phòng** ($V_{BAT}$): Có dải từ 1.8V đến 3.6V, dùng để duy trì bộ RTC và các thanh ghi dự phòng (BKP) khi nguồn $V_{DD}$ chính bị ngắt.

### 1.2. Bộ điều áp nguồn (Voltage Regulator) và các trạng thái hoạt động

Bộ điều chỉnh điện áp luôn được kích hoạt sau khi reset. Trong STM32F103C8T6 nó đóng vai trò quan trọng trong việc kiểm soát năng lượng tại miền 1.8V thông qua ba chế độ hoạt động sau:

-Trong Run mode: Bộ điều áp cung cấp đầy đủ năng lượng để các ngoại vi hoạt động ở công suất cao nhất.

-Trong Stop mode: Bộ điều chỉnh cung cấp điện năng thấp cho miền 1,8 V, giữ nguyên nội dung của các thanh ghi và SRAM.

-Trong Standby mode: Bộ điều áp bị ngắt hoàn toàn. Đầu ra của nó chuyển sang trạng thái trở kháng cao, làm cho miền 1.8V không còn điện năng và dẫn đến mức tiêu thụ dòng điện cực thấp (chỉ khoảng 2 µA ở 3.3V).

# CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VỀ SLEEP MODE

## 2.1. Tổng quan về chế độ Sleep Mode

-Chế độ Sleep là trạng thái tiết kiệm năng lượng cơ bản nhất trong các chế độ công suất thấp của vi điều khiển STM32F103C8T6. Đặc trưng cốt lõi của chế độ này là sự phân tách rạch ròi giữa các thành phần phần cứng tạm dừng hoạt động và các thành phần duy trì trạng thái làm việc. Trong suốt chu kỳ Sleep, thành phần duy nhất bị đưa vào trạng thái không hoạt động (Inactive) là lõi xử lý trung tâm CPU Cortex-M3. Việc ngừng thực thi lệnh tại lõi giúp cắt giảm phần lớn công suất tiêu thụ động của hệ thống, đồng thời tạo ra một khoảng nghỉ cần thiết khi các tác vụ tính toán đã hoàn tất.

-Mặc dù CPU tạm dừng, các thành phần quan trọng khác của hệ thống như miền nguồn kỹ thuật số 1.8V, bộ điều áp chính (Main Regulator), bộ nhớ Flash và SRAM vẫn được duy trì ở trạng thái hoạt động (Active). Sự tồn tại liên tục của nguồn điện trong các miền này đảm bảo ngữ cảnh thực thi của chương trình và toàn bộ dữ liệu lưu trữ trong các thanh ghi chức năng không bị mất đi. Điều này cho phép vi điều khiển có khả năng phục hồi tức thì mà không cần trải qua quy trình tái khởi động hệ thống phức tạp, giúp tối ưu hóa thời gian phản hồi và duy trì tính ổn định cho các ứng dụng yêu cầu xử lý liên tục.

Ngoài hệ thống nguồn và bộ nhớ, các ngoại vi số và tương tự như bộ định thời (Timers), bộ chuyển đổi ADC hay các cổng giao tiếp vẫn có thể được cấu hình để tiếp tục vận hành bình thường trong chế độ Sleep. Khả năng duy trì hoạt động của các ngoại vi cho phép vi điều khiển giám sát các sự kiện từ môi trường bên ngoài một cách độc lập với lõi xử lý. Khi một ngắt hoặc sự kiện xảy ra, hệ thống sẽ ngay lập tức kích hoạt lõi CPU trở lại trạng thái hoạt động. Chính sự phối hợp nhịp nhàng giữa việc tạm dừng lõi và duy trì sự sống cho các thành phần hệ thống khác đã tạo nên ưu thế của Sleep Mode trong việc cân bằng giữa hiệu suất tiết kiệm năng lượng và khả năng đáp ứng thời gian thực

## 2.2. Kiến trúc điều khiển năng lượng tại lõi Cortex-M3

Kiến trúc quản lý năng lượng của STM32F103C8T6 không chỉ phụ thuộc vào các miền nguồn ngoại vi mà còn được điều khiển trực tiếp từ cấu trúc bên trong của lõi Cortex-M3. Lõi này tích hợp các cơ chế quản lý năng lượng thông qua khối điều khiển hệ thống, cho phép phần mềm can thiệp sâu vào việc dừng hoặc kích hoạt lại xung nhịp CPU. Điểm cốt lõi của kiến trúc này là sự phối hợp giữa bộ quản lý ngắt NVIC (Nested Vectored Interrupt Controller) và các chế độ nghỉ được cấu hình sẵn, giúp hệ thống có thể thoát khỏi trạng thái tiết kiệm điện một cách nhanh chóng và chính xác khi có tín hiệu từ bên ngoài.

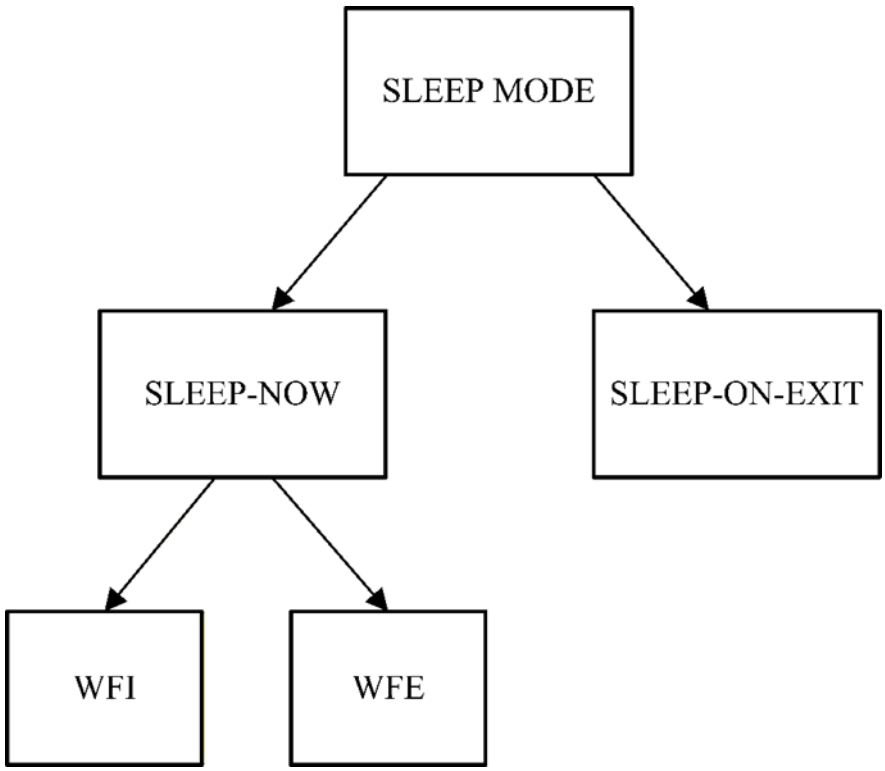
### *2.2.1. Thanh ghi điều khiển hệ thống (System Control Register - SCR)*

-Thanh ghi SCR là trung tâm điều phối các trạng thái năng lượng thấp thuộc khối điều khiển hệ thống của Cortex-M3. Việc cấu hình chế độ Sleep hay Deep Sleep được quyết định bởi các bit chức năng trong thanh ghi này, trong đó quan trọng nhất là bit SLEEPDEEP. Để thực hiện chế độ Sleep thông thường, bit SLEEPDEEP phải được giữ ở mức 0. Khi đó, hệ thống chỉ thực hiện ngắt xung nhịp lõi CPU mà không tác động đến bộ điều áp hay các miền nguồn quan trọng, đảm bảo dữ liệu trong SRAM được bảo toàn và thời gian phục hồi là ngắn nhất.

### *2.2.2. Vai trò của bit SLEEPONEXIT trong việc quản lý chu trình ngủ*

-Bit SLEEPONEXIT trong thanh ghi SCR mang lại khả năng tự động hóa quy trình tiết kiệm năng lượng cho các ứng dụng thực tế. Khi bit này được thiết lập bằng 1, vi điều khiển sẽ tự động quay trở lại chế độ Sleep ngay sau khi kết thúc trình phục vụ ngắt (ISR) mà không cần thực thi lại luồng chương trình chính (Main loop). Cơ chế này đặc biệt tối ưu cho các hệ thống hoạt động theo sự kiện (Interrupt-driven), giúp loại bỏ các chu kỳ CPU dư thừa cho việc quản lý ngăn xếp (Stacking/Unstacking) khi thoát khỏi ngắt, từ đó giảm thiểu dòng điện tiêu thụ trung bình và tăng hiệu suất vận hành của toàn hệ thống.

## 2.3. Các cơ chế kích hoạt Sleep Mode




Hình 2: Các chế độ Sleep Mode

-Việc triển khai Sleep Mode trên dòng STM32F103 được xây dựng dựa trên sự phối hợp giữa trạng thái phần cứng và logic điều phối của phần mềm ứng dụng. Cơ chế này thiết lập một trạng thái vận hành đặc thù, trong đó lõi xử lý trung tâm được tạm dừng hoàn toàn để cắt giảm năng lượng tiêu thụ, trong khi các khối ngoại vi và hệ thống nguồn vẫn duy trì trạng thái hoạt động bình thường. Đặc điểm này cho phép vi điều khiển bảo toàn toàn bộ dữ liệu trong bộ nhớ RAM và các thanh ghi chức năng, đồng thời đảm bảo các thành phần ngoại vi vẫn có thể giám sát tín hiệu để sẵn sàng đánh thức hệ thống ngay khi có sự kiện xảy ra.

-Dựa trên yêu cầu của người lập trình, Sleep Mode được phân chia thành các chế độ vận hành cụ thể tùy thuộc vào thời điểm vi điều khiển đi vào trạng thái nghỉ. Sự phân cấp này định nghĩa kịch bản thực thi lệnh để CPU dừng hoạt động, giúp tối ưu

hóa chu kỳ làm việc của bộ vi xử lý và đảm bảo lõi xử lý chỉ tiêu thụ năng lượng khi thực sự có tác vụ cần giải quyết.

### *2.3.1. Cơ chế Sleep Now:*

-Cơ chế Sleep Now cho phép hệ thống đi vào trạng thái nghỉ ngay lập tức sau khi các lệnh điều khiển chuyên biệt được thực thi trong luồng chương trình chính. Đây là kịch bản vận hành chủ động, thường được áp dụng khi các tác vụ xử lý nền đã kết thúc và hệ thống cần chờ đợi một tín hiệu từ môi trường bên ngoài. Để thực hiện cơ chế này, người lập trình sử dụng một trong hai lệnh hợp ngữ của lõi Cortex-M3:

- Lệnh WFI (Wait For Interrupt): Khi lệnh này được gọi, CPU sẽ dừng hoạt động và chỉ thức dậy khi nhận được một tín hiệu ngắt (Interrupt) có mức ưu tiên đủ cao từ NVIC.

- Lệnh WFE (Wait For Event): Khác với WFI, lệnh này đưa CPU vào trạng thái nghỉ và cho phép đánh thức hệ thống thông qua một sự kiện (Event) hoặc ngắt, giúp tối ưu hóa thời gian phản hồi cho các ứng dụng không cần xử lý ngắt phức tạp.

### *2.3.2. Cơ chế Sleep on Exit:*

-Đối với các ứng dụng vận hành hoàn toàn dựa trên sự kiện (Interrupt-driven), cơ chế Sleep on Exit mang lại hiệu quả tiết kiệm năng lượng sâu hơn. Để sử dụng cơ chế này, người lập trình cần thiết lập bit SLEEPONEXIT (bit thứ 1) trong thanh ghi điều khiển hệ thống (System Control Register - SCR) của lõi Cortex-M3 lên mức 1.

-Khi bit này được kích hoạt, vi điều khiển sẽ tự động quay trở lại trạng thái Sleep ngay sau khi thoát khỏi trình phục vụ ngắt (ISR) mà không cần quay lại thực thi các lệnh trong chương trình chính. Bằng cách loại bỏ các chu kỳ xử lý không cần thiết và giảm bớt các thao tác quản lý ngăn xếp (Stacking/ Unstacking) khi thoát khỏi ngắt, Sleep on Exit giúp giảm thiểu dòng điện tiêu thụ trung bình, đặc biệt hiệu quả cho các hệ thống cảm biến chỉ cần thức dậy để xử lý dữ liệu nhanh chóng rồi lập tức trở lại trạng thái nghỉ.

## 2.4. Lệnh thực thi và phương thức đánh thức

-Hoạt động của Sleep Mode trên lõi Cortex-M3 được điều khiển trực tiếp bởi tập lệnh chuyên biệt nhằm tối ưu hóa sự cân bằng giữa tiết kiệm điện năng và tốc độ đáp ứng. Việc lựa chọn lệnh thực thi và cơ chế đánh thức không chỉ quyết định dòng tiêu thụ mà còn ảnh hưởng đến trình tự xử lý của vi điều khiển sau khi thoát khỏi trạng thái nghỉ.

### *2.4.1. Lệnh WFI (Wait For Interrupt)*

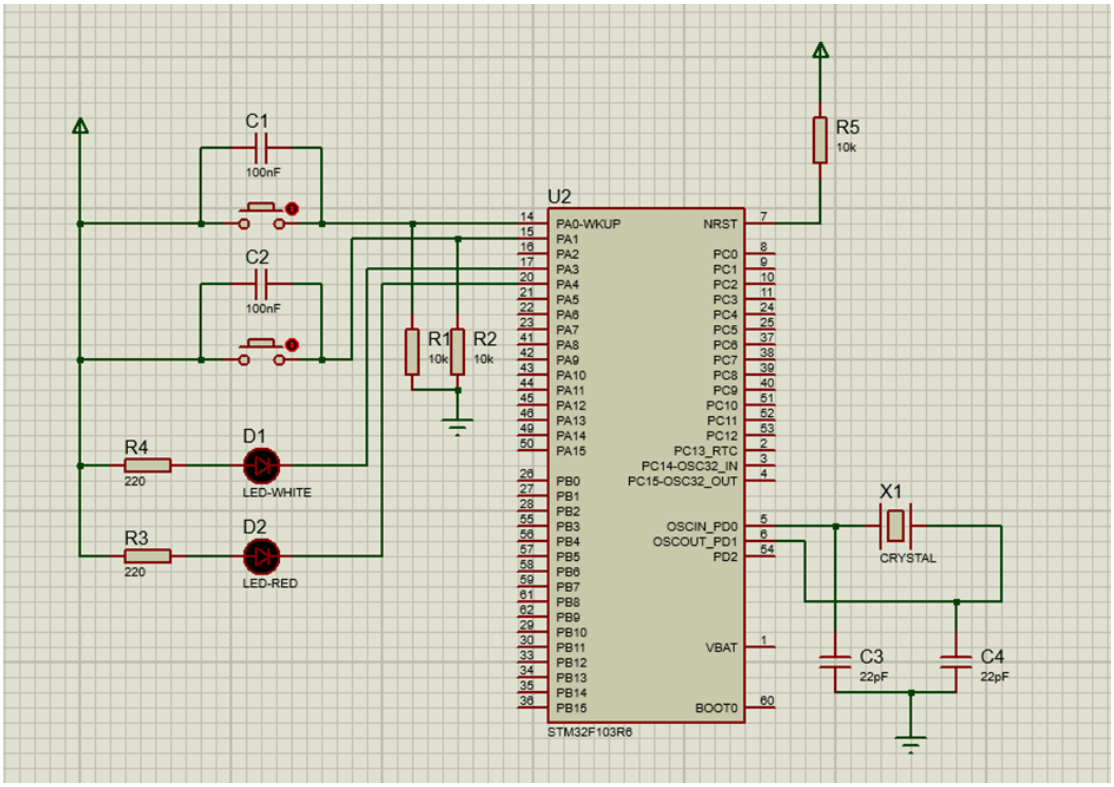
-Lệnh WFI là phương thức phổ biến nhất để đưa hệ thống vào trạng thái Sleep. Khi lệnh này được thực thi, lõi CPU sẽ ngay lập tức tạm dừng việc lấy lệnh và giải mã, đồng thời đi vào trạng thái chờ đợi. CPU sẽ chỉ được đánh thức khi có một yêu cầu ngắt từ NVIC (Nested Vectored Interrupt Controller) có mức ưu tiên đủ cao. Một đặc điểm quan trọng của cơ chế đánh thức bằng WFI là sau khi nhận tín hiệu, CPU sẽ nhảy vào trình phục vụ ngắt (ISR) để xử lý tác vụ trước, sau đó mới quay lại thực hiện các lệnh tiếp theo ngay sau vị trí lệnh WFI trong chương trình chính.

### *2.4.2. Lệnh WFE (Wait For Event)*

-Khác với WFI, lệnh WFE cho phép vi điều khiển nghỉ và chờ đợi một sự kiện thay vì chỉ phụ thuộc vào ngắt. Một sự kiện có thể được tạo ra bởi tín hiệu từ các ngoại vi hoặc thông qua việc thiết lập bit SEVONPEND trong thanh ghi điều khiển. Ưu điểm cốt lõi của WFE là nếu hệ thống được đánh thức bởi một sự kiện đang chờ xử lý, CPU có khả năng tiếp tục thực thi các lệnh tiếp theo ngay lập tức mà không nhất thiết phải nhảy vào trình phục vụ ngắt. Điều này giúp giảm thiểu độ trễ phản hồi và tiết kiệm các chu kỳ máy dư thừa cho các ứng dụng đòi hỏi tốc độ đáp ứng cực nhanh.

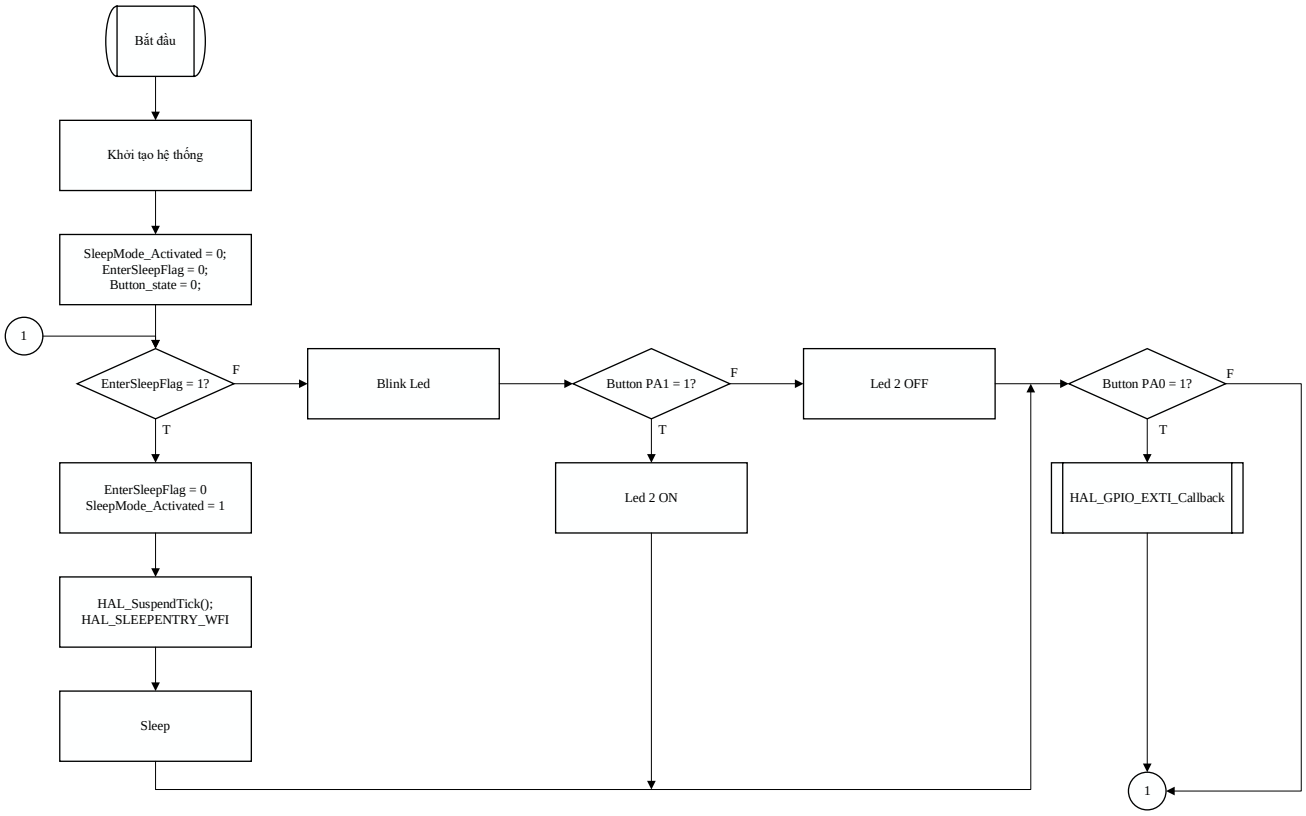
# CHƯƠNG 3: THIẾT KẾ CHƯƠNG TRÌNH THỰC NGHIỆM SLEEP MODE

## 3.1. Sơ đồ kết nối phần cứng




Hình 3: Sơ đồ nối chân bằng Proteues

### 3.2. Lưu đồ thuật toán




Hình 4: Lưu đồ thuật toán chương trình Sleep Mode

- Lưu đồ chương trình Sleep Mode trên vi điều khiển STM32F103C8T6 thực hiện theo các bước như sau:

+ Bước 1: Khởi tạo hệ thống, set trạng thái bạn đầu cho các cờ (Flag) và các biến cục bộ

```
uint8_t SleepMode_Activated = 0;   // Flag: CPU is in sleep
uint8_t EnterSleepFlag = 0;        // Flag: request to enter sleep
uint8_t Button_state = 0;
```

+ Bước 2: Kiểm tra cờ EnterSleepFlag có bằng 1 hay không

Nếu cờ EnterSleepFlag = 1, thì:

EnterSleepFlag = 0; → set cờ EnterSleepFlag cho lần sau.

SleepMode_Activated = 1; → set cờ SleepMode_Activated lên 1 để biết hệ thống đang ở Sleep Mode.

HAL_SuspendTick(); →Lệnh vô hiệu hoá ngắt của hệ thống SysTick.

HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_O , PWR_SLEEPENTRY_WFI); → Lệnh dùng để duy trì bộ điều áp chính và set lệnh chờ ngắt (WFI) cho hệ thống.

Nếu cờ EnterSleepFlag = 0, thì thực hiện bước tiếp theo.

+ Bước 3: Thực hiện hàm Blink Led theo chu trình như hình 3-3

+ Bước 4: Kiểm tra Button PA1 (Pull Down)

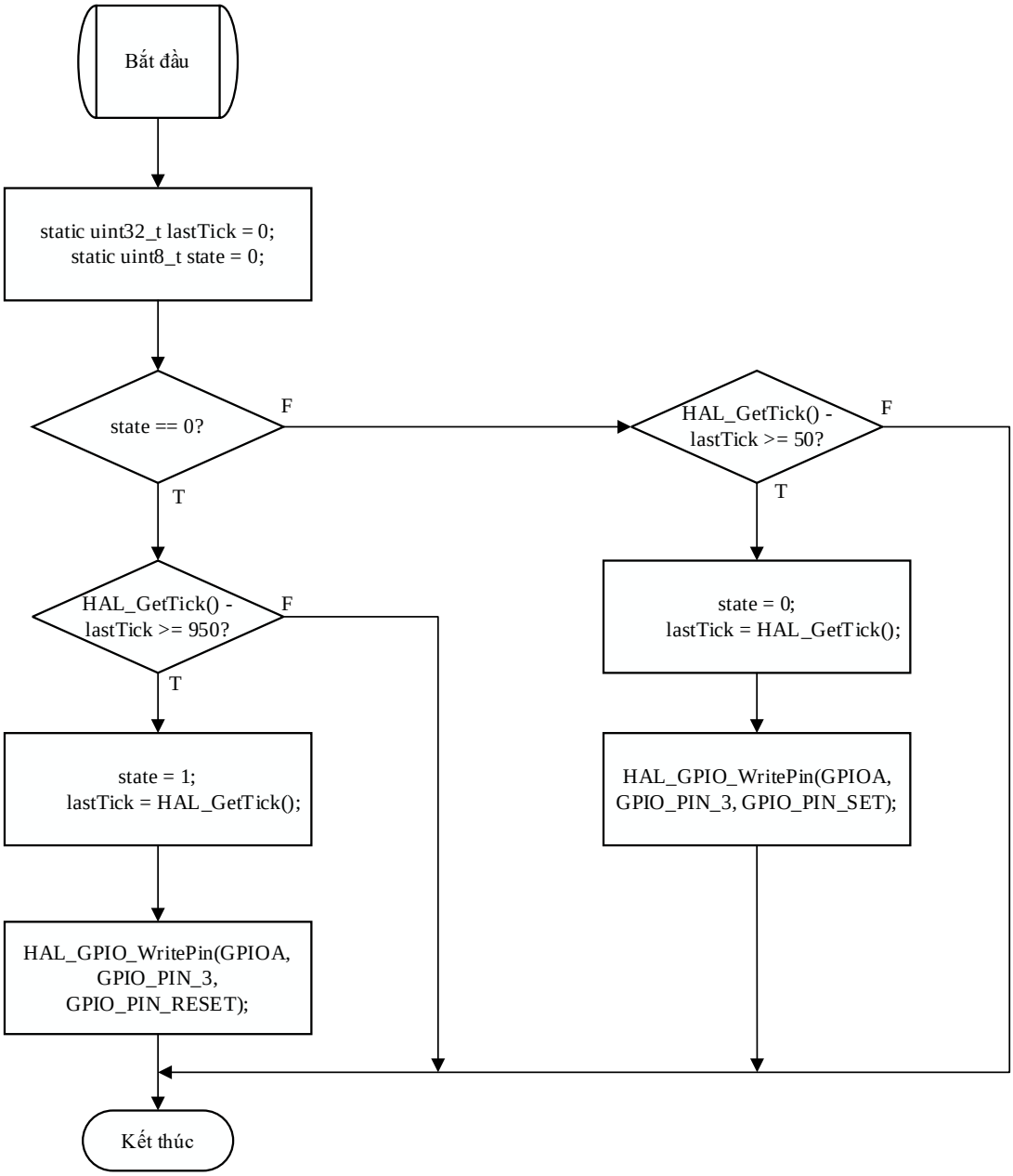
Nếu Button PA1 = 1, thì Led 2 sẽ sáng.

Nếu Button PA1 = 0, thì Led 2 sẽ tắt.

+ Bước 5: Kiểm tra Button PA0 (Pull Down)

Nếu Button PA0 = 1, thì nhảy vào chương trình ngắt HAL_GPIO_EXTI_Callback có chu trình như hình 3-4.

Nếu Button PA0 = 0; thì quay lại vị trí 1 và tiếp tục lặp lại các bước trên.




Hình 5: Lưu đồ thuật toán của chương trình Blink Led

- Lưu đồ chương trình Blink Led nằm bên trong hàm main sẽ thực hiện theo các bước như sau:

Bước 1: set giá trị ban đầu cho các biến

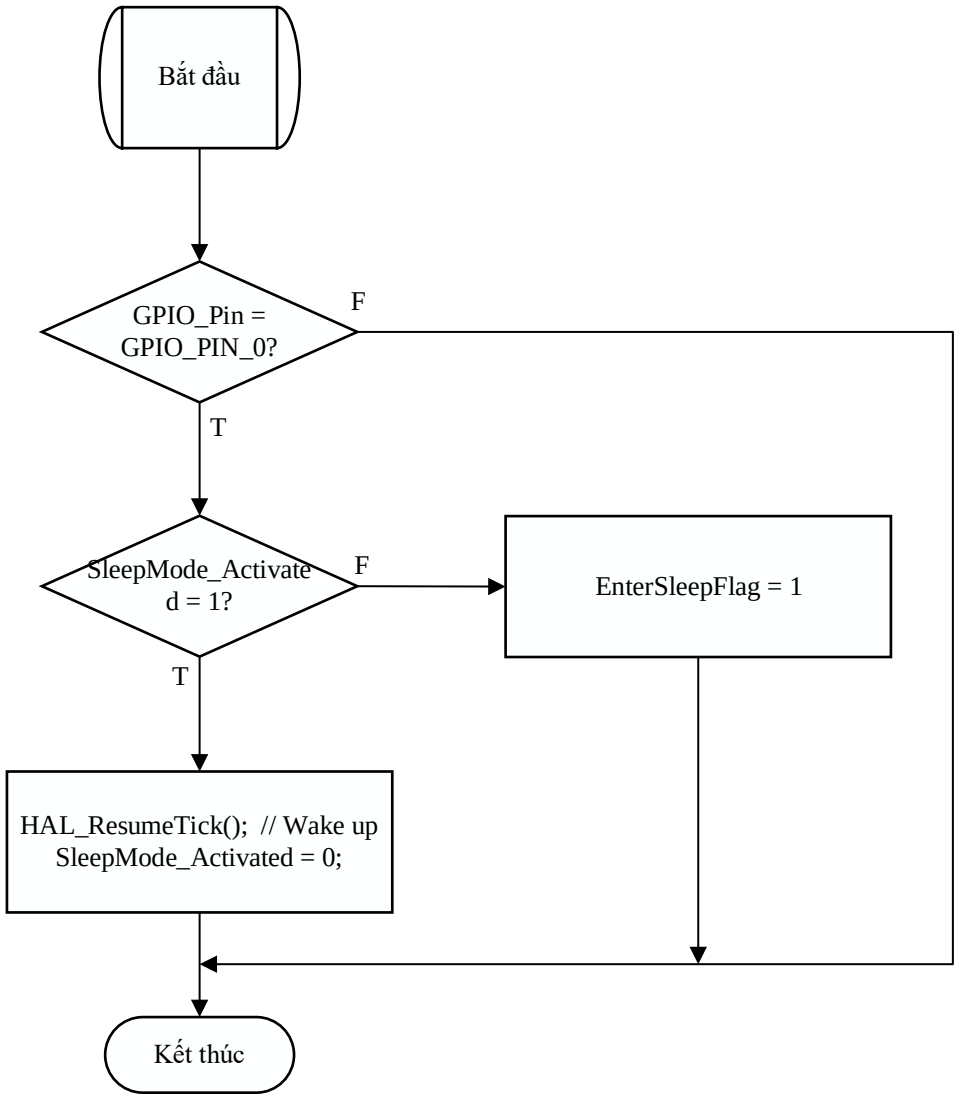
static uint32_t lastTick = 0; Dùng để lưu lại mốc thời gian cuối cùng mà LED thay đổi trạng thái.

static uint8_t state = 0; Dùng để lưu trạng thái hiện tại của máy trạng thái LED (LED đang trong giai đoạn Tắt hay giai đoạn Bật).

Bước 2: Kiểm tra biến state

State = 0; LED đang tắt và đang chờ đợi đủ 950ms để bật lên.

State = 1; LED đang sáng và đang chờ đủ 50ms để tắt đi.




Hình 6: Lưu đồ thuật toán của chương trình ngắt

- Lưu đồ chương trình ngắt CALLBACK nằm bên trong hàm main sẽ thực hiện theo các bước như sau:

+ Bước 1: Kiểm tra xem có đúng là tín hiệu ngắt từ chân GPIO_PIN_0 hay không.

Nếu đúng, thì tiếp tục thực hiện các bước sau.

Nếu sai, thì kết thúc chương trình.

+ Bước 2: Kiểm tra biến SleepMode_Activated xem hệ thống có đang trong Sleep Mode hay không.

Nếu SleepMode_Activated = 1 (Hệ thống đang trong chế độ Sleep), thì thực hiện bước tiếp theo.

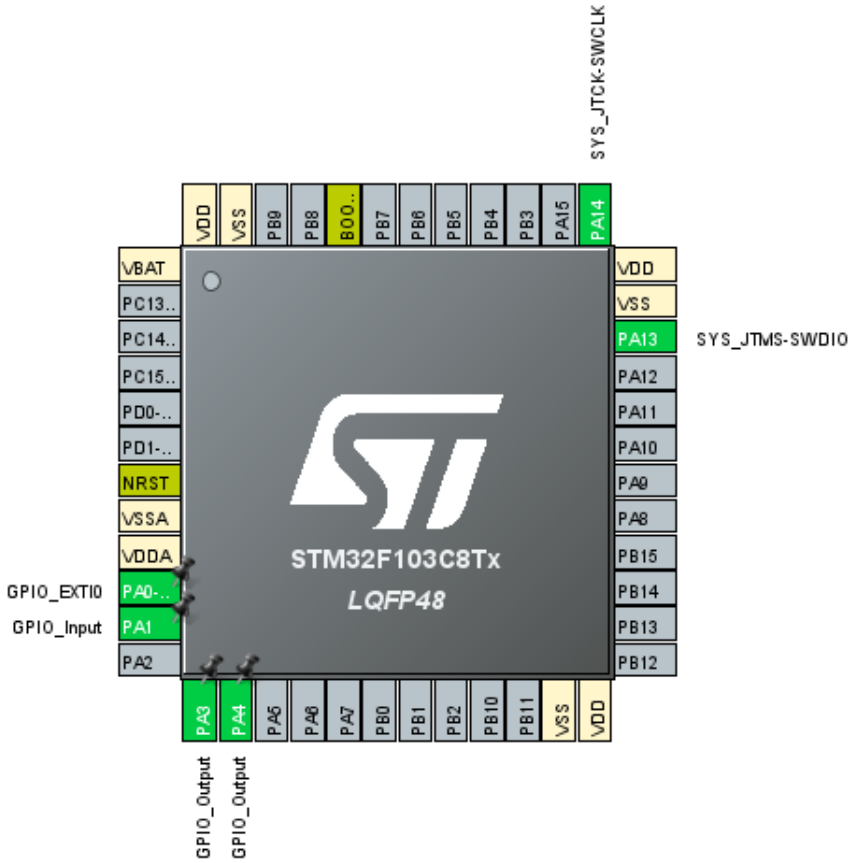
Nếu SleepMode_Activated = 0 (Hệ thống đang trong chế độ Run), thì set cờ EnterSleepFlag = 1 sau đó kết thúc chương trình.

+ Bước 3: set lại các biến để hệ thống thoát khỏi chế độ Sleep

HAL_ResumeTick();  đánh thức hệ thống.

SleepMode_Activated = 0; set lại biến để dùng cho lần sau.

### 3.3. Cấu hình CubeMX




Hình 7: Cấu hình chân STM32F103C8T6

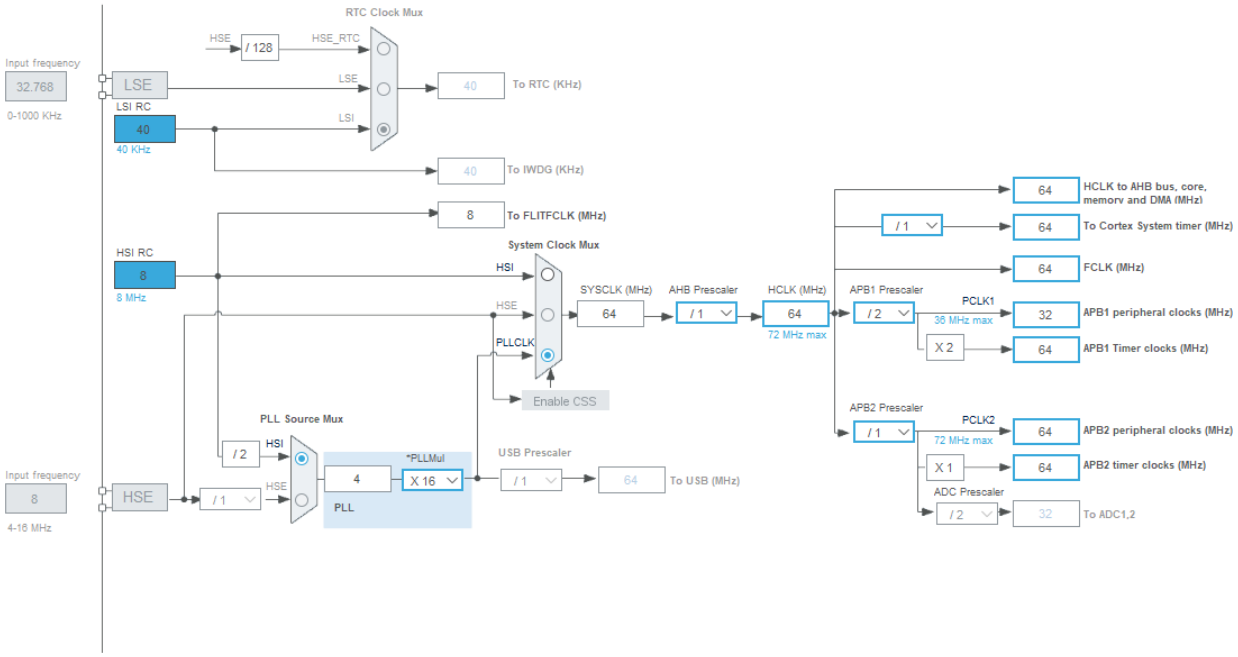
Cấu hình cho STM32F103C8T6 bao gồm:

+) Chân PA0: ngắt để vào và thoát Sleep Mode.

+) Chân PA1: nút điều khiển led PA3.

+) Chân PA3: GPIO cho GPIO_PIN_3.

+) Chân PA4: GPIO cho GPIO_PIN_4.




Hình 8: Cấu hình xung CLK STM32F103C8T6

-Cấu hình hệ thống sử dụng mạch vòng khóa pha (PLL) với nguồn cấp từ dao động nội HSI, thiết lập xung nhịp hệ thống tại mức 64MHz. Trong trạng thái Sleep, việc duy trì mạch PLL và các hệ thống bus APB1/APB2 hoạt động ở tần số cao đảm bảo các ngoại vi giám sát (như Timer và EXTI) có độ phân giải thời gian cực cao. Tuy nhiên, mức xung nhịp này cũng xác định một 'ngưỡng nền' về tiêu thụ năng lượng, do các bộ dao động cao tần không bị ngắt hoàn toàn như trong các chế độ Stop hay Standby.

### 3.4. Phân tích chương trình

Code chính:

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
```

```
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2026 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"


/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */


/* USER CODE END Includes */


/* Private typedef -----------------------------------------------------------*/
```

```
/* USER CODE BEGIN PTD */



/* USER CODE END PTD */



/* Private define ------------------------------------------------------------*/

/* USER CODE BEGIN PD */

uint8_t SleepMode_Activated = 0;   // Flag: CPU is in sleep

uint8_t EnterSleepFlag = 0;        // Flag: request to enter sleep

uint8_t Button_state = 0;

/* USER CODE END PD */



/* Private macro -------------------------------------------------------------*/

/* USER CODE BEGIN PM */



/* USER CODE END PM */



/* Private variables ---------------------------------------------------------*/



/* USER CODE BEGIN PV */



/* USER CODE END PV */
```

```c
/* Private function prototypes -----------------------------------------------*/

void SystemClock_Config(void);

static void MX_GPIO_Init(void);

/* USER CODE BEGIN PFP */



/* USER CODE END PFP */



/* Private user code ---------------------------------------------------------*/

/* USER CODE BEGIN 0 */

void blink_led(void)

{

    static uint32_t lastTick = 0;

    static uint8_t state = 0;



    if (state == 0) { // LED OFF

        if (HAL_GetTick() - lastTick >= 950) {

            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_RESET);

            state = 1;

            lastTick = HAL_GetTick();

        }

    } else { // LED ON

        if (HAL_GetTick() - lastTick >= 50) {
```

```
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET);
            state = 0;
            lastTick = HAL_GetTick();
        }
    }
}
/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick.
```

```
*/
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */
       HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET);
  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  /* USER CODE BEGIN 2 */

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
```

```
while (1) {

            if (EnterSleepFlag)

  {

      EnterSleepFlag = 0;

      SleepMode_Activated = 1;

      HAL_SuspendTick();

      HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);

  }



            Button_state = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1);



  if (Button_state == GPIO_PIN_SET)

  {

      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET);

  }

  else

  {

      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);

  }



            blink_led();
```

```
    /* USER CODE END WHILE */


    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}


/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};


  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
```

```
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI_DIV2;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL16;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}
```

```c
/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOA_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3|GPIO_PIN_4, GPIO_PIN_RESET);

  /*Configure GPIO pin : PA0 */
  GPIO_InitStruct.Pin = GPIO_PIN_0;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
```

```
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pin : PA1 */
  GPIO_InitStruct.Pin = GPIO_PIN_1;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : PA3 PA4 */
  GPIO_InitStruct.Pin = GPIO_PIN_3|GPIO_PIN_4;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /* EXTI interrupt init*/
  HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(EXTI0_IRQn);

  /* USER CODE BEGIN MX_GPIO_Init_2 */
```

```
  /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
/**
  * @brief EXTI line callback for PA0
  */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_Pin == GPIO_PIN_0) // Sleep control button
    {
        if(SleepMode_Activated == 1) {
            HAL_ResumeTick();       // Wake up
            SleepMode_Activated = 0;
        } else {
            EnterSleepFlag = 1;     // Enter sleep on next loop
        }
    }
}
/* USER CODE END 4 */

/**
```

```
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
```

```
{

  /* USER CODE BEGIN 6 */

  /* User can add his own implementation to report the file name and line
number,

     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */

  /* USER CODE END 6 */

}

#endif /* USE_FULL_ASSERT */
```

- Phân tích Code:

-Hàm Blink Led:

```
void blink_led(void)

{

    static uint32_t lastTick = 0;

    static uint8_t state = 0;



    if (state == 0) { // LED OFF

        if (HAL_GetTick() - lastTick >= 950) {

            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_RESET);

            state = 1;

            lastTick = HAL_GetTick();
```

```
        }
    } else { // LED ON
        if (HAL_GetTick() - lastTick >= 50) {
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET);
            state = 0;
            lastTick = HAL_GetTick();
        }
    }
}
```

+ Hàm được gọi liên tục trong while(1)

+ led sẽ sáng trong 50ms và tắt trong 950ms

- Luồng chính trong while(1):

```
while (1) {
            if (EnterSleepFlag)
    {
        EnterSleepFlag = 0;
        SleepMode_Activated = 1;
        HAL_SuspendTick();
        HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
    }
```

```
            Button_state = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1);


    if (Button_state == GPIO_PIN_SET)
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET);
    }
    else
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);
    }


            blink_led();
```

- Cấu hình xung hệ thống:

```
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
```

```
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI_DIV2;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL16;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
```

```
  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}
```

- Cấu hình GPIO:

```
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOA_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3|GPIO_PIN_4, GPIO_PIN_RESET);

  /*Configure GPIO pin : PA0 */
```

```
GPIO_InitStruct.Pin = GPIO_PIN_0;

GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;

GPIO_InitStruct.Pull = GPIO_NOPULL;

HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);



/*Configure GPIO pin : PA1 */

GPIO_InitStruct.Pin = GPIO_PIN_1;

GPIO_InitStruct.Mode = GPIO_MODE_INPUT;

GPIO_InitStruct.Pull = GPIO_NOPULL;

HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);



/*Configure GPIO pins : PA3 PA4 */

GPIO_InitStruct.Pin = GPIO_PIN_3|GPIO_PIN_4;

GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;

GPIO_InitStruct.Pull = GPIO_NOPULL;

GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);



/* EXTI interrupt init*/

HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0);

HAL_NVIC_EnableIRQ(EXTI0_IRQn);
```

```
  /* USER CODE BEGIN MX_GPIO_Init_2 */



  /* USER CODE END MX_GPIO_Init_2 */

}
```

- Hàm ngắt CALLBACK:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

{

    if(GPIO_Pin == GPIO_PIN_0) // Sleep control button

    {

        if(SleepMode_Activated == 1) {

            HAL_ResumeTick();       // Wake up

            SleepMode_Activated = 0;

        } else {

            EnterSleepFlag = 1;     // Enter sleep on next loop

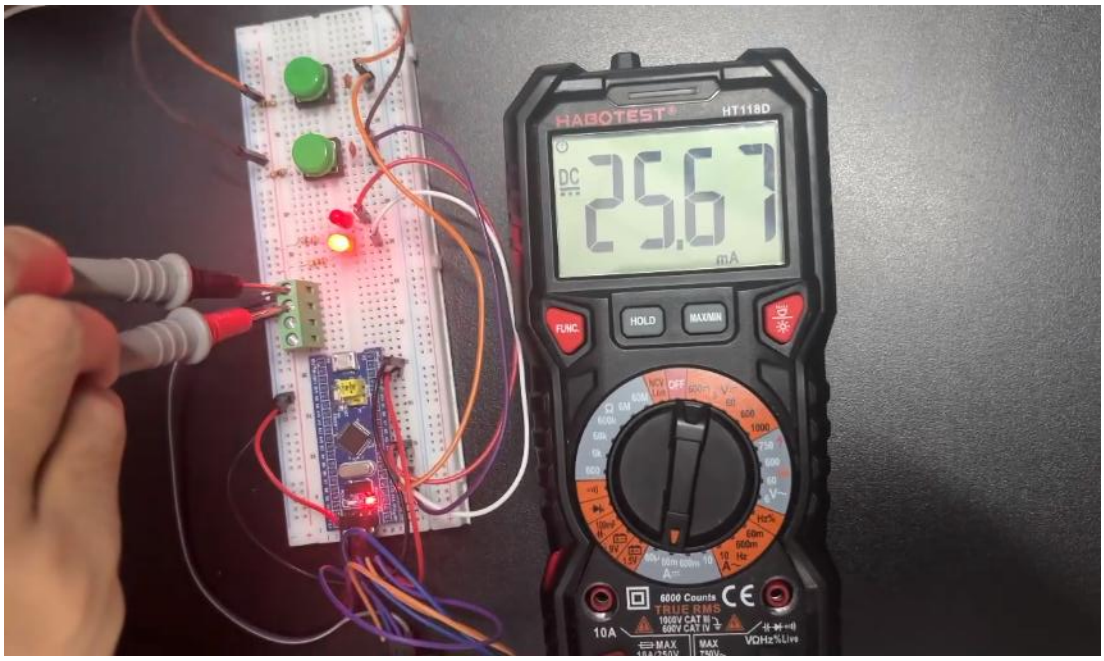
        }

    }

}
```

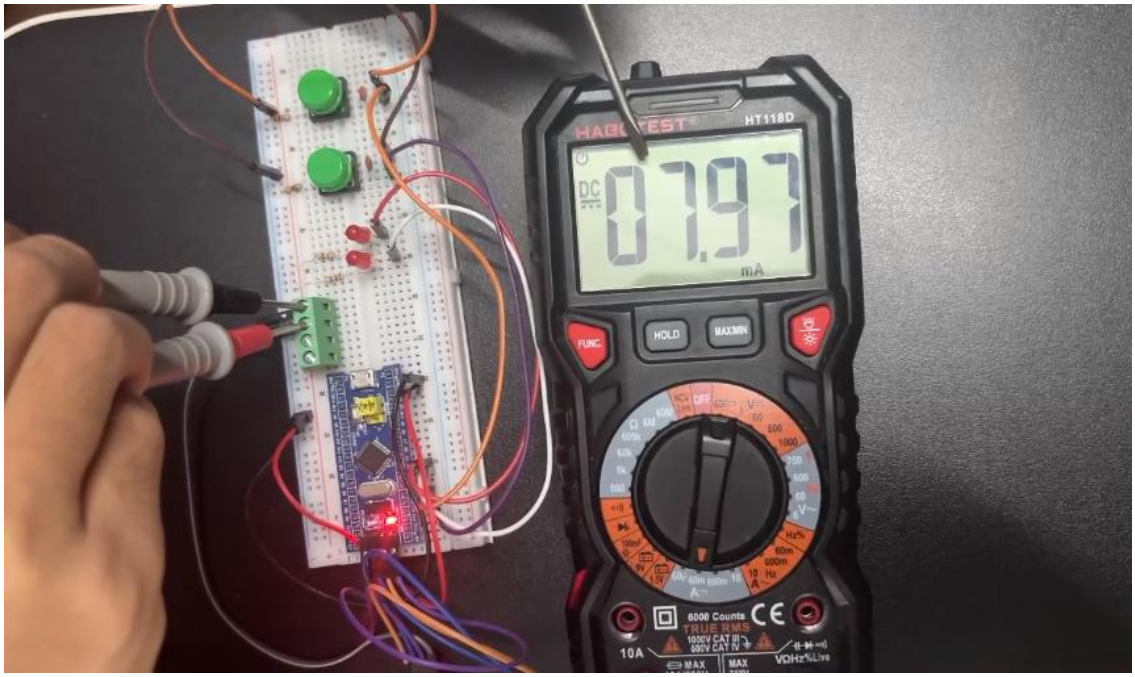
# CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH

## 4.1. Kết quả đo lường tiêu thụ dòng điện.



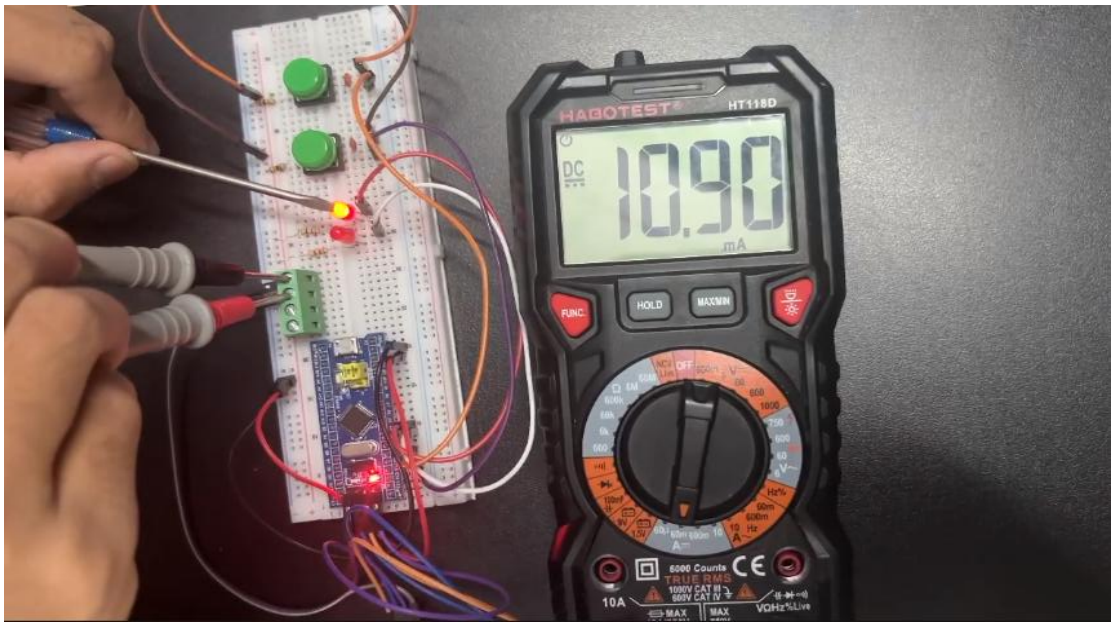

Hình 9: Đo dòng điện ở chế độ Run

Khi hệ thống ở chế độ Run Mode thì dòng điện khoảng 25.67mA.




Hình 10: Đo dòng điện ở chế độ Sleep

Khi hệ thống đã rơi vào trạng thái Sleep Mode thì dòng điện chỉ còn khoảng 7.97mA. Thấp hơn khá nhiều so với chế độ Sleep Mode đo được ở trên.




Hình 11: Đo dòng điện ở chế độ Sleep có GPIO ở mức High

Khi hệ thống đã rơi vào trạng thái Sleep Mode nhưng có một chân GPIO giữ mức logic 1 (HIGH) thì dòng điện lúc này khoảng 10.9mA.

### 4.2. Phân tích kết quả đo lường do điện tiêu thụ

Theo datasheet STM32F103C8T6 ở chế độ Sleep Mode có dòng tiêu thụ chỉ ở mức khoảng 5.5mA, nhưng khi đo trên bo mạch thực tế Blue Pill với STLink rời, dòng tiêu thụ đạt khoảng 7.97 mA. Nguyên nhân chính là do LED nguồn trên bo mạch.

- LED nguồn được nối trực tiếp với đường 3.3 V thông qua điện trở hạn dòng và luôn sáng theo trạng thái hoạt động của STM32. Với điện áp rơi thuận của LED khoảng 2 V, dòng tiêu thụ ước tính vào khoảng 1–1.3 mA, làm thay đổi giá trị thực tế so với lý thuyết. Và khi có chân GPIO giữ mức logic 1 thì dòng điện cũng có sự tăng nhẹ.

- Từ đây có thể đánh giá dòng điện trong chế độ Sleep Mode phụ thuộc vào số ngoại vi còn hoạt động.

# CHƯƠNG 5: KẾT LUẬN VÀ ĐÁNH GIÁ

## 5.1. Ưu và nhược điểm của Sleep Mode

- Ưu điểm:

Phản hồi tức thì: Thời gian thức tỉnh cực ngắn (vài chu kỳ xung nhịp) nhờ không phải khởi động lại các bộ dao động thạch anh.

Bảo toàn dữ liệu: Giữ nguyên 100% nội dung RAM và các thanh ghi. Các biến static (lastTick, state) duy trì giá trị liên tục, không cần cấu hình lại phần mềm sau khi ngủ.

Ngoại vi linh hoạt: Timer, ADC, UART vẫn chạy bình thường, cho phép hệ thống giám sát tín hiệu ngay cả khi CPU đang nghỉ.

Triển khai dễ dàng: Sử dụng lệnh HAL đơn giản, không phức tạp như các chế độ Stop hay Standby.

- Nhược điểm:

Tiết kiệm điện thấp nhất: Là chế độ tiêu thụ dòng điện cao nhất trong các chế độ Low-power vì hệ thống xung nhịp ngoại vi vẫn hoạt động.

Quản lý thủ công: Người dùng phải tự tắt các ngoại vi không dùng đến để đạt mức tiêu thụ điện tối ưu.

Nhạy cảm với ngắt: CPU dễ bị thức tỉnh bởi các nguồn ngắt không mong muốn (ví dụ: nhiễu nút nhấn) nếu không được lọc kỹ.

## 5.2. Đánh giá hiệu quả của chế độ Sleep Mode

- Tiết kiệm năng lượng: dù không phải là chế độ tiết kiệm điện sâu nhất (như Stop hay Standby), Sleep Mode vẫn mang lại hiệu quả rõ rệt so với chế độ chạy bình thường (Run Mode).

- Hiệu quả bảo toàn dữ liệu: Toàn bộ dữ liệu trên RAM và các thanh ghi cấu hình không bị mất đi.

- Hiệu quả về thời gian phản hồi: Sleep Mode cung cấp khả năng "thức dậy" nhanh nhất trong các chế độ Low-power. Vì hệ thống chỉ mất khoảng 6 chu kỳ xung nhịp để quay lại trạng thái Run.

### 5.3. Các ứng dụng thực tế

Trong thực tiễn kỹ thuật, Sleep Mode được ứng dụng rộng rãi trong các hệ thống đòi hỏi sự cân bằng khắt khe giữa hiệu suất năng lượng và tính thời gian thực. Đặc biệt đối với các thiết bị đo lường và giao tiếp dữ liệu liên tục, khả năng duy trì hoạt động của hệ thống xung nhịp ngoại vi (PCLK) trong khi đình chỉ lõi xử lý cho phép giảm dòng tiêu thụ mà vẫn đảm bảo tính sẵn sàng cao của hệ thống.

## TÀI LIỆU THAM KHẢO

[1] STMicroelectronics, *AN2629: STM32F101xx, STM32F102xx and STM32F103xx lowpower modes*, Doc. ID 13922, Rev. 2, Apr. 2009.

[2] STMicroelectronics, *STM32F103x8, STM32F103xB Datasheet – Medium-density performance line Arm®-based 32-bit MCU*, DS5319, Rev. 20, Jul. 2025.

[3] STMicroelectronics, *RM0008: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx Advanced ARM-based 32-bit MCUs – Reference manual*, Doc. ID 13902, Rev. 9, Jun. 2009.

**[4]** Next Electronics, *ARM Cortex-M Series Architecture Tutorial*, online tutorial. Available:***https://www.next.gr/tutorials/microcontrollers-and-developmentboards/arm-cortex-m-series-architecture-tutorial***